

C# Læringsprosjekt

Dependency Injection · Entity Framework · Arkitektur · Testing · Service Bus
ElektronikkButikk Web API

Innledning

Dette dokumentet er en gjennomgang av alt vi har bygd i løpet av dette læringsprosjektet. Vi har gått fra grunnleggende konsepter til et fullstendig Web API med database, forretningslogikk, meldingssystem og automatiske tester.

Prosjektet er delt i to deler: et lite læringsprosjekt (CodingCourse) og et fullstendig Web API (ElektronikkButikk). Begge bruker de samme prinsippene, men ElektronikkButikk er mer realistisk og nærmere hva du ville bygd i en faktisk jobb.

Konsept	Hva det handler om
Dependency Injection (DI)	Klasser ber om avhengigheter istedenfor å lage dem selv
Entity Framework (EF)	Snakk med databasen uten å skrive SQL for hånd
Arkitekturvalg	Lagdelt monolitt med Repository og Service-lag
Testrammeverk	Automatiske tester med xUnit og NSubstitute
Service Bus	Løst koblet meldingsutveksling mellom systemer

Del 1: Dependency Injection (DI)

Dependency Injection er grunnmuren for alt vi har bygd. Det handler om ett enkelt prinsipp: klasser skal ikke lage sine egne avhengigheter — de skal be om dem utenfra.

Uten DI (tett koblet)

Uten DI lager en klasse sine avhengigheter selv med `new`. Dette gjør koden tett koblet og vanskelig å teste:

```
public class AppRunner
{
    private readonly FriendlyGreetingService _service;

    public AppRunner()
    {
        _service = new FriendlyGreetingService(); // lager den selv – TETT KOBLET
    }
}
```

```
}
```

Problemet: AppRunner er hardkoblet til FriendlyGreetingService. Vil du bytte implementasjon, må du inn og endre selve klassen.

Med DI (løs kobling)

Med DI ber klassen om det den trenger via konstruktøren. En DI-container løser avhengighetene automatisk:

```
public class AppRunner
{
    private readonly IGreetingService _service;

    public AppRunner(IGreetingService service) // ber om det den trenger
    {
        _service = service;
    }
}
```

AppRunner bryr seg ikke om hvilken implementasjon den får — det er DI-containerens jobb.

Interface — kontrakten

Et interface beskriver HVA en tjeneste gjør, ikke HVORDAN. Vi programmerer alltid mot interfacet, aldri mot den konkrete klassen:

```
public interface IGreetingService
{
    string Greet(string name); // kontrakten
}

public class FriendlyGreetingService : IGreetingService
{
    public string Greet(string name)
    {
        return $"Hei {name}!"; // implementasjonen
    }
}
```

ServiceCollection — DI-containeren

ServiceCollection er oppskriftsboken der vi registrerer alle tjenester og kobler interfaces til implementasjoner:

```
var services = new ServiceCollection();
services.AddTransient();
var serviceProvider = services.BuildServiceProvider();
```

Lifetime — hvor lenge lever en instans?

Lifetime	Forklaring
Transient	Ny instans HVER gang noen ber om den. Brukes til lett logikk og stateless tjenester.
Scoped	Samme instans innenfor én HTTP-request. Brukes til DbContext og database-operasjoner.
Singleton	Én instans for hele applikasjonens levetid. Brukes til cache og konfigurasjon.

Del 2: Entity Framework (EF)

Entity Framework er et ORM (Object Relational Mapper). Det betyr at du jobber med vanlige C#-klasser istedenfor å skrive SQL for hånd. EF oversetter alt til SQL automatisk.

Model — en vanlig C#-klasse

En EF-model er en helt vanlig C#-klasse. EF ser på den og lager automatisk en tabell i databasen:

```
public class Product
{
    public int Id { get; set; }           // EF gjenkjenner Id som primærnøkkel
    public string Name { get; set; } = string.Empty;
    public decimal Price { get; set; }
    public int StockQuantity { get; set; }
}
```

DbContext — broen til databasen

DbContext er hjertet i Entity Framework. Den holder styr på alle endringer og oversetter dem til SQL:

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions options)
        : base(options) { }

    public DbSet Products { get; set; } = null!;
    // DbSet representerer en tabell i databasen
}
```

EF-operasjoner og SQL-ekvivalenter

C#-kode	SQL som kjøres
context.Products.ToListAsync()	SELECT * FROM Products
context.Products.FindAsync(id)	SELECT * FROM Products WHERE Id = @id
context.Products.Add(product)	Markerer for INSERT (ikke sendt ennå)
context.SaveChangesAsync()	Sender INSERT/UPDATE/DELETE til databasen
.Include(o => o.Customer)	JOIN med Customer-tabellen
.ThenInclude(i => i.Product)	Ytterligere JOIN ett nivå dypere

Relasjoner mellom tabeller

EF håndterer relasjoner via navigasjonsproperties og fremmednøkler:

```
public class Order
{
    public int CustomerId { get; set; }    // Fremmednøkkel
    public Customer Customer { get; set; } // Navigasjonsproperty
    public List<Item> Items { get; set; } = new();
}
```

EF ser CustomerId og Customer-propertyen og lager automatisk en fremmednøkkel i databasen som kobler Orders til Customers.

Del 3: Arkitekturvalg — Lagdelt Monolitt

Vi valgte en monolitt med god intern struktur, delt inn i klare lag. Dette gir oss fordelene av enkelhet, men med en arkitektur som kan deles opp til microservices senere hvis det trengs.

Lagene i arkitekturen

Lag	Ansvar
Controllers	Tar imot HTTP-requests og sender svar tilbake
Services	Forretningslogikk og regler — HER bor reglene
Repositories	Henter og lagrer data i databasen
Models	Datastrukturene som flyter gjennom systemet
ServiceBus	Sender meldinger om hendelser i systemet

Dataflyten i en request — POST /api/Orders

```
HTTP Request (POST /api/Orders)
↓
OrdersController.CreateOrder() ← tar imot og validerer input
↓
OrderService.CreateOrder() ← sjekker forretningsregler
↓
ProductRepository.GetById() ← henter produkt fra database
↓
ProductRepository.Update() ← oppdaterer lagerbeholdning
↓
OrderRepository.Add() ← lagrer bestillingen
↓
ServiceBus.Publish() ← sender OrderCreatedMessage
↓
HTTP Response (201 Created)
```

Hvorfor Repository-laget?

Repository-laget er et skjold mellom resten av appen og databasen. All SQL-logikk er samlet ett sted. Hvis du bytter fra SQLite til PostgreSQL, er det bare Repository-laget du trenger å endre.

Hvorfor Service-laget?

Service-laget er der reglene bor. Repository vet bare HVORDAN man lagrer data — Service vet OM man skal lagre det, og under hvilke betingelser:

```
public async Task AddProduct(string name, string brand, decimal price, int stock)
{
    if (price <= 0)
        throw new ArgumentException("Pris kan ikke være negativ eller null");
    if (stock < 0)
        throw new ArgumentException("Lagerbeholdning kan ikke være negativ");
    // Forretningsregler hører hjemme her – ikke i Repository
}
```

Del 4: Testrammeverk

Tester er det som gjør at vi trygt kan endre kode uten å være redde for å ødelegge noe. God arkitektur med interfaces og DI gjør testing enkelt — vi kan bytte ut ekte avhengigheter med fakes.

Verktøyene vi brukte

Verktøy	Hva det gjør
xUnit	Testrammeverket. [Fact] markerer en test-metode.
NSubstitute	Lager fake versjoner av interfaces (mocks).
Received(1)	Sjekker at en metode ble kalt nøyaktig én gang.
DidNotReceive()	Sjekker at en metode IKKE ble kalt.
Assert.ThrowsAsync()	Sjekker at koden kaster riktig exception.
Returns()	Forteller en fake hva den skal returnere.

Hvorfor kan vi teste uten en database?

Fordi vi programmerte mot interfaces! I testene bytter vi ut den ekte ProductRepository med en fake versjon — ingen database, ingen SQL, bare ren logikk:

```
public ProductServiceTests()
{
    _fakeRepository = Substitute.For();
    _fakeServiceBus = Substitute.For();
    // Ingen ekte database – bare attrapper
}
```

```

        _productService = new ProductService(_fakeRepository, _fakeServiceBus);
        // ProductService vet ikke at de er fake – den bryr seg ikke
    }

```

Eksempel på en test — Arrange, Act, Assert

```

[Fact]
public async Task UpdateStock_ShouldPublishLowStockMessage_WhenStockBelow5()
{
    // Arrange – sett opp testdata
    var product = new Product { Id = 1, Name = 'MacBook Pro', StockQuantity = 10 };
    _fakeRepository.GetById(1).Returns(product);

    // Act – kjør koden vi tester
    await _productService.UpdateStock(1, 3); // 3 er under 5

    // Assert – sjekk at det riktige skjedde
    _fakeServiceBus.Received(1).Publish(Arg.Is(m =>
        m.ProductId == 1 && m.RemainingStock == 3));
}

```

Del 5: Service Bus

En service bus løser problemet med tett kobling mellom systemer. Istedenfor at en tjeneste kaller en annen direkte, sender den en melding på bussen. Andre systemer kan lytte og reagere når de er klare.

Uten vs. med Service Bus

	Forklaring
Uten Service Bus	ProductService kaller LagerService direkte. Hvis LagerService er nede, krasjer alt.
Med Service Bus	ProductService sender en melding på bussen og går videre. LagerService leser meldingen når den er klar.

Meldingene vi bruker

Melding	Sendes når
OrderCreatedMessage	En bestilling blir lagt inn. Inneholder OrderId, CustomerId og TotalAmount.
LowStockMessage	Lagerbeholdningen faller under 5. Inneholder ProductId, ProductName og RemainingStock.

Implementasjonen

Vi programmerte mot et interface — så det er enkelt å bytte fra InMemoryServiceBus til Azure Service Bus:

```

public interface IServiceBus
{

```

```
        void Publish(T message); // = kan sende hvilken som helst melding
    }

    public class InMemoryServiceBus : IServiceBus
    {
        public void Publish(T message)
        {
            Console.WriteLine($"[ServiceBus] Melding sendt: {typeof(T).Name}");
            // I virkeligheten: send til Azure Service Bus eller RabbitMQ
        }
    }
}
```

ElektronikkButikk Web API

ElektronikkButikk er et fullstendig ASP.NET Core Web API som bruker alle de fem konseptene i praksis. Her er en oversikt over hele prosjektstrukturen og hvordan alt henger sammen.

Prosjektstruktur

ElektronikkButikk/	
Models/	
Product.cs	← Elektronikkprodukt med navn, merke, pris og lager
Customer.cs	← Kunde med navn, e-post og telefon
Order.cs	← Bestilling med OrderItems
Data/	
AppDbContext.cs	← EF-broen til databasen
Repositories/	
IProductRepository	← Kontrakt for produktdata
ProductRepository	← Henter/lagrer produkter
ICustomerRepository	← Kontrakt for kundedata
CustomerRepository	← Henter/lagrer kunder
IOrderRepository	← Kontrakt for bestillingsdata
OrderRepository	← Henter/lagrer bestillinger
Services/	
IProductService	← Kontrakt for produktlogikk
ProductService	← Validering og forretningsregler for produkter
ICustomerService	← Kontrakt for kundelogikk
CustomerService	← Validering og forretningsregler for kunder
IOrderService	← Kontrakt for bestillingslogikk
OrderService	← Lager bestillinger og oppdaterer lager
Controllers/	
ProductsController	← HTTP-endepunkter for produkter
CustomersController	← HTTP-endepunkter for kunder
OrdersController	← HTTP-endepunkter for bestillinger
ServiceBus/	
IServiceBus	← Kontrakt for meldingssystemet
InMemoryServiceBus	← Simulert implementasjon
Messages.cs	← OrderCreatedMessage, LowStockMessage

API-endepunktene

Endepunkt	Hva det gjør
GET /api/Products	Henter alle produkter fra databasen
GET /api/Products/{id}	Henter ett produkt basert på ID
POST /api/Products	Legger til et nytt produkt
PUT /api/Products/{id}/stock	Oppdaterer lagerbeholdningen
DELETE /api/Products/{id}	Sletter et produkt
GET /api/Customers	Henter alle kunder

GET /api/Customers/{id}	Henter én kunde med bestillingshistorikk
POST /api/Customers	Registrerer en ny kunde
GET /api/Orders	Henter alle bestillinger med produktdetaljer
GET /api/Orders/{id}	Henter én bestilling
POST /api/Orders	Oppretter en ny bestilling

DI-registrering i Program.cs

Her er alt som registreres i DI-containeren — dette er der alle brikkene kobles sammen:

```
builder.Services.AddDbContext(options =>
    options.UseSqlite('Data Source=elektronikk.db'));
// Scoped av EF som standard
builder.Services.AddScoped();
builder.Services.AddScoped();
builder.Services.AddScoped();
// Scoped = samme instans per HTTP-request
builder.Services.AddScoped();
builder.Services.AddScoped();
builder.Services.AddScoped();
builder.Services.AddSingleton();
// Singleton = én delt instans for hele appen
```

Testene — 5 av 5 bestått

Test	Hva den sjekker
AddProduct_ShouldCallAdd_WhenValid	At Add() kalles med riktig produkt når alt er gyldig
AddProduct_ShouldThrow_WhenPriceIsNegative	At en negativ pris kaster ArgumentException
AddProduct_ShouldThrow_WhenStockIsNegative	At en negativ lager kaster ArgumentException
UpdateStock_ShouldPublishLowStockMessage_WhenStockBelow5	At en LowStockMessage publiseres når lager er under 5
UpdateStock_ShouldNotPublishMessage_WhenStockAbove5	At ingen LowStockMessage publiseres når lager er over 5

Oppsummering — hva henger sammen?

Alt vi har lært henger tett sammen. Her er det store bildet:

- Interfaces gjør koden løst koblet og testbar
- DI kobler interfaces til implementasjoner og injiserer dem automatisk
- EF bruker DbContext (registrert med DI) til å snakke med databasen
- Repository-laget isolerer databaselogikken fra resten av appen
- Service-laget inneholder forretningsreglene og bruker DI til å få Repository
- Controllers tar imot HTTP-requests og bruker DI til å få Service

- Service Bus sender meldinger løst koblet mellom systemer
- Tester bruker NSubstitute til å bytte ut ekte avhengigheter med fakes — mulig pga. interfaces

Dette er de samme prinsippene du vil møte i ASP.NET Core, Azure-integrasjoner og enterprise-applikasjoner. Du har et solid grunnlag å bygge videre på!